A Verified Static Analyzer for a Toy Imperative Language

Fahim A. Islam

ECS 261 – Program Verification
University of California, Davis
Winter 2026

1 Introduction, Problem Selection, and Goals

Ensuring that programs execute safely without runtime failures is a central concern in software development. Runtime errors are common sources of crashes and undefined behavior. Traditional testing, such as unit tests, can reveal the presence of such bugs on selected inputs, but it cannot guarantee their absence for all possible executions. Static analysis addresses this limitation by reasoning about program behavior *before* runtime.

Static analysis tools are widely used in compilers, integrated development environments (IDE's), and security scanners to detect bugs early in development. However, most practical analyzers are implemented and tested rather than formally verified. As a result, while such tools may be useful in practice, they usually do not come with machine-checked guarantees that the analyzer itself is correct. This motivates the study of *verified static analyzers*, where the analyzer implementation is formally proven sound.

In this project, I implemented and formally verified a static analyzer for a small imperative programming language in **Dafny**. The analyzer targets two classes of runtime errors:

- division by zero
- use of uninitialized variables.

This project is conceptually similar to verified static analyzers such as Verasco, which proves soundness of an abstract interpreter for the C language. Unlike Verasco, which operates on large real-world programs, this project focuses on a simplified imperative language to demonstrate the key proof ideas.

The design also draws inspiration from the abstract interpretation framework [1], in which concrete program states are approximated by an abstract domain sufficient to reason about the safety property of interest [1]. Instead of tracking exact integer values, the analyzer tracks abstract information about variables such as whether they are zero, positive, negative, or unknown. This abstraction allows the analyzer to reason about many possible executions using finite information while remaining sound.

The central correctness property established in this project is a soundness theorem of the following form:

> If the analyzer determines that a program is safe, then executing that program cannot produce a division-by-zero or uninitialized-variable runtime error.

In Dafny, this theorem is encoded as the lemma `AnalyzeProgramSound`, which formally proves that programs accepted by the analyzer cannot produce runtime errors.

This project also addresses feedback from the earlier project proposal. In particular, the final version includes a clearer formal soundness goal, concrete analyzer rules, and explicit discussion of how the proof relates to known static-analysis techniques such as abstract interpretation.

Project Goals

I had several goals for this project split into minimum goals, target goals, and reach goals.

Minimum goals:

- Define toy imperative language syntax with basic math operations and statement structure.
- Define runtime semantics of the language.
- Implement static analyzer for rudimentary runtime error detection, prioritizing soundness.

Target goals:

- Implement abstract interpretation for static analysis.
- Prove that the analyzer is sound with respect to the runtime semantics using Dafny lemmas.
- Demonstrate the analyzer on multiple safe and unsafe example programs.

Reach goals:

- Extend abstract domain past NonZero, Zero, Unknown.
- Expand language functionality past basic mathematical operations conditionals. (e.g. while loops, conditionals, etc.)
- Generalize the analyzer to a greater scope of errors.

The final implementation achieved all the minimum and target goals. It partially achieved the reach goals by extending the abstract domain to track positive and negative values and by adding a simplified counted-loop construct.

2 Implementation

2.1 Tool Selection

The entire project was implemented in **Dafny**, a verification-aware programming language and program verifier designed for proving functional correctness properties [2]. Dafny was a natural choice for this project because it provides:

- datatypes for defining the abstract syntax tree of the language

- lemmas for structuring proofs
- automated proof support through the Z3 SMT solver.

Verification was performed using Dafny's built-in verifier. In addition, I created a GitHub repository to manage versions of the code and tests:

https://github.com/FahimIslam731/Static-Analyzer

All source files, proofs, and tests were written in Dafny. No external implementation language was used.

2.2 Architecture of the Code

The final code is organized into three main source files:

- `Types.dfy`: datatype definitions for expressions, statements, abstract values, runtime errors, results, environments,
- `StaticAnalyzer.dfy`: runtime semantics, abstract evaluation, static analysis, and proof lemmas;
- `Tests.dfy`: example test programs used to validate the implementation and demonstrate analyzer behavior.

Conceptually, the system has four main components.

(1) Language syntax.
The toy language contains expressions and statements. Expressions include constants, variables, addition, subtraction, multiplication, division, and modulo. Statements include skip, assignment, sequential composition, conditionals, and a simplified counted loop.

(2) Runtime semantics.
Concrete program execution is modeled by two functions:

- `EvalExpr(expression, environment)` evaluates an expression under a concrete environment.
- `ExecStmt(statement, environment)` executes a statement under a concrete environment. `ExecStmt` will call `EvalExpr` when a statement contains an expression.

These functions define the reference behavior of programs, including the possibility of runtime errors.

(3) Abstract interpretation layer.
The analyzer uses an abstract state:

$$\texttt{AbsState} = \texttt{map<string, AbsVal>}$$

where `AbsVal` $\in$ `{Zero, Positive, Negative, Unknown}`. This domain is a simple sign abstraction, a common technique used in abstract interpretation [1].

This abstraction approximates sign information about program values. The analyzer computes an abstract interpretation of expressions using the `AbsEval(expression, state)` function.

(4) Proof layer.
The proof layer connects the abstract analysis with the concrete semantics. The key invariant is a predicate `Consistent(state, environment)` stating that the abstract state correctly reflects the sign information of the concrete environment. Proof lemmas establish that:

1. Expressions marked safe by the analyzer cannot produce runtime errors.
   `lemma CheckTrueEvalValid(expression, state, environment)`
2. Statements preserve consistency between abstract states and concrete environments.
   `lemma AnalyzeTrueExecValid(statement, state, environment)`
3. Therefore, programs accepted by the analyzer cannot produce the targeted runtime errors.

```
lemma AnalyzeProgramSound(prog, environment)
      requires analyze_program(prog, environment)
      ensures !IsErr(exec_stmt(prog, environment))
```

Implemented vs. unimplemented features.
The final code fully implements and verifies:

- arithmetic expressions
- assignments
- sequential composition
- conditionals with abstract-state joining
- a simplified counted loop
- the core soundness proof

I did not implement features such as:

- boolean expressions and comparisons
- general while-loops with fixpoint iteration
- taint analysis

2.3 Verification Effort

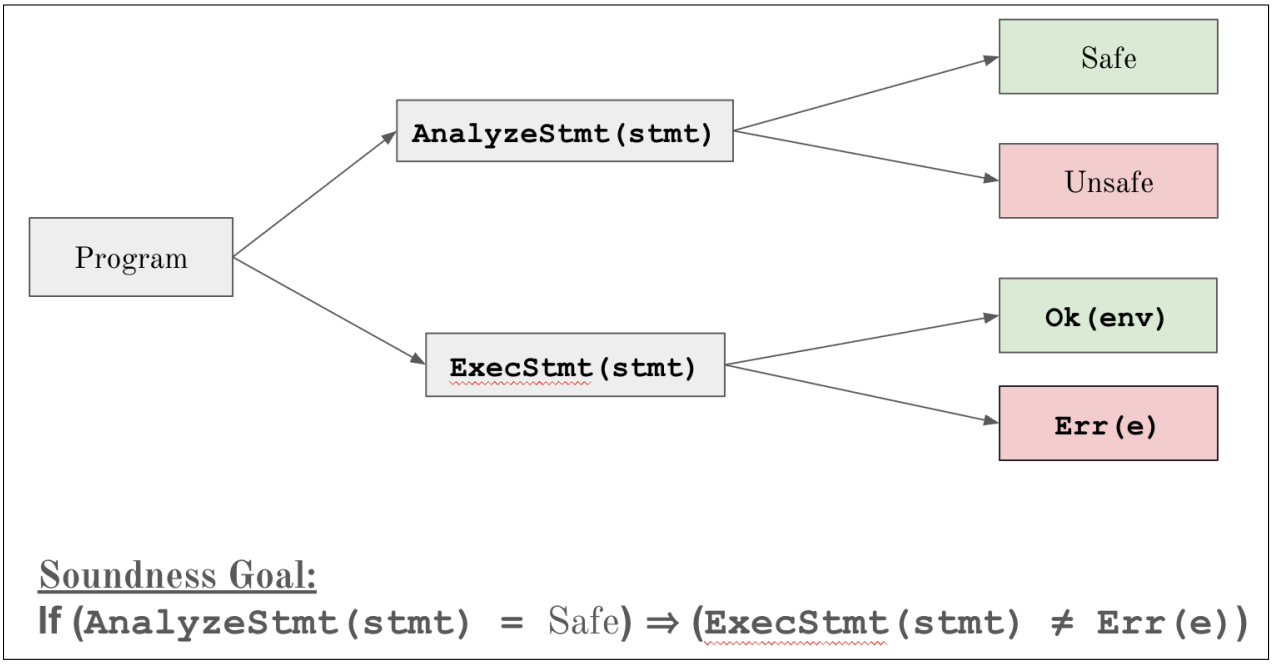

Figure 1: High-level architecture of the verified static analyzer

The main theorem proved in the project is the soundness theorem for the analyzer. At a high level, it states:

> If `AnalyzeStmt` marks a program safe, then executing that program with `ExecStmt` cannot produce either `DivByZero` or `UninitializedVar`.

The proof effort focused on the following components:

Expression-level verification.
I proved that if `CheckExpr(e, st)` returns true and the abstract state is consistent with the runtime environment, then evaluating `e` cannot produce a runtime error. This required helper lemmas relating abstract values to concrete values in the runtime environment, such as:

- if abstract evaluation says an expression is `Zero`, then its concrete value is 0;
- if abstract evaluation says an expression is `Positive`, then its concrete value is > 0;
- if abstract evaluation says an expression is `Negative`, then its concrete value is < 0.

The main lemma used for this task was `lemma CheckTrueEvalValid(expression, state, environment)`.

Statement-level verification.
The main statement-level lemma is proved by induction on the structure of statements. It shows that if a statement is accepted by the analyzer, then:

- the resulting concrete environment remains consistent with the analyzer's resulting abstract state.

- executing the statement cannot produce a runtime error.

The main lemma used for this task was `lemma AnalyzeTrueExecValid(statement, state, environment)`, which uses `lemma Consistent(state, environment)` as a postcondition.

Conditional statements.
Conditionals required proving that the join operator on abstract states preserves soundness. This led to the creation of helper lemmas which show that if either branch state is consistent with the runtime environment, then the joined state is also consistent.

The main lemmas used for this task were `lemma ConsistentJoinLeft(state_true, state_false, environment)` and `lemma ConsistentJoinRight(state_true, state_false, environment)`

Loop statements.
The loop extension was verified under a conservative rule: the analyzer only accepts a loop if one iteration of the body is safe and leaves the abstract state unchanged. This effectively turns the abstract state into a loop invariant, which makes the proof tractable.

The main lemma used for this task was `lemma ExecLoopPreservesInvariant(runtimes, loop_body, state, environment)`

All verification was performed at *compile time* by the Dafny verifier. There are no runtime proof checks.

3 Challenges

As is certainly a rite of passage with verification projects, several challenges arose during the course of this one.

3.1 Balancing Precision and Verifiability

One of the main design challenges was choosing an abstract domain that was expressive enough to detect the target runtime errors while still being simple enough to verify. The initial design used a domain with only `Zero`, `NonZero`, and `Unknown`. This was sufficient for detecting the runtime errors of concern (preserving soundness), but it was too imprecise for some arithmetic reasoning. For example, expressions involving addition and subtraction could lose useful sign information.

To improve precision, I extended the domain to use:

```
Zero, Positive, Negative, Unknown
```

This made the analyzer more expressive, but it also increased the complexity of the proofs because more cases had to be handled every lemma that dealt with all the potential cases of the abstract domain.

3.2 Proof Size and Explicit Guidance

Although the core proof idea is simple to describe, the actual Dafny proof is much longer than the runtime implementation. A large part of the difficulty came from the fact that the SMT solver often requires explicit guidance. Facts that seem mathematically obvious (e.g. how sign information

should propagate through arithmetic cases) still needed to be encoded via explicit assertions and lemmas.

In practice, the project required several hundred lines of proof code. The proof structure is clean, but the verifier still needed substantial case splitting and bridging lemmas.

3.3 Conditionals and State Joining

The `If` case was one of the most involved parts of the analyzer. When the condition expression evaluates abstractly to `Unknown`, the analyzer must analyze both branches and join the resulting states. Proving soundness for this join operation required separate lemmas showing that if either branch matches the concrete runtime state, then the joined state also remains sound. This was conceptually simple but subtle to formalize.

3.4 Loops

General loops are difficult for abstract interpretation because they typically require fixpoint computation and widening [1, 3]. To keep the extension manageable within the scope of the project, I implemented a simplified counted-loop construct and used a conservative analysis rule: the body must preserve the same abstract state after one iteration. This made the loop extension verifiable, but also very restrictive.

3.5 Unverified Assumptions

A strength of the final project is that I did *not* rely on `assume` statements, axioms, or unproven correctness assumptions in the final code.

4 Results

4.1 Implementation Results

On the implementation side, the final project supports:

- arithmetic expressions over integers,
- assignments,
- sequential composition,
- conditional branching,
- a simplified counted loop,
- detection of division-by-zero and uninitialized-variable errors.

The repository contains several test cases showing analyzer behavior on representative programs. A few examples are summarized below.

Example 1: Safe arithmetic example

```
v := 4 + 4;
v := v / 2;
```

The analyzer accepts this program, and execution produces a valid environment in which $v = 4$.

Example 2: Unsafe division-by-zero example

```
z := 1 / 0;
```

The analyzer rejects this program, and the runtime semantics produce `DivByZero`.

Example 3: Conditional pruning example

```
if (1) {
    //condition will be true if it doesn't evaluate to 0.
    x := 1;
} else {
    y := 1 / 0;
}
z := x;
```

Because the condition is definitely positive, the analyzer only needs to analyze the then-branch. The program is accepted and executes safely.

Example 4: Incompleteness.
A test case used in the presentation and report demonstrates that the analyzer is sound but not complete. For example, if the analyzer only knows that two operands are nonzero or have uncertain sign interactions, it may classify a condition as `Unknown` even when runtime execution determines a definite sign. In such cases, safe programs may still be rejected.

4.2 Verification Results

On the verification side, the primary "QED" is `AnalyzeProgramSound`, which proves:

> If the analyzer marks a program as safe, then evaluating that program cannot produce either a division-by-zero or uninitialized-variable error.

This theorem is supported by several lower-level proven properties:

- `CheckExpr` implies expression-level safety;
- `AnalyzeStmt` preserves consistency between abstract and concrete states;
- join operations for conditional branches preserve soundness;
- the counted-loop rule is sound under the invariant-preservation restriction.

At a high level, the analyzer is therefore:

- **Sound**, because any program it accepts is guaranteed not to trigger the targeted runtime errors.
- **Incomplete**, because some safe programs rejected due to the conservative nature of the analyzer.

4.3 Goals Revisited

The final outcome relative to the goals from Section 1 is as follows.

Minimum goal.
Achieved. The language, runtime semantics, and baseline analyzer were fully implemented.

Target goal.
Achieved. The analyzer uses an abstract domain and is formally proven sound in Dafny.

Reach goal.
Partially achieved. The abstract domain was extended to track sign information and a simplified loop construct was added. Fully functional while loops, taint analysis, and richer domains were not implemented.

4.4 Effort and Size

The project was developed over roughly the full project timeline of the quarter. At a rough level:

- early work focused on language design and runtime semantics,
- middle-stage work focused on implementing abstract interpretation and expression analysis,
- late-stage work focused heavily on proof lemmas, conditionals, and loop extensions.

The code base is on the order of several hundred lines of Dafny. The proof code constitutes a large fraction of the overall project effort, significantly exceeding the size of the executable semantics themselves.

5 Contribution Statement

Contributors

This project was completed individually by, me, Fahim A. Islam. I designed the toy language, implemented the runtime semantics, developed the abstract domain and static analyzer, proved the soundness lemmas in Dafny, and created the proposal, presentation, tests, and final report. I also iteratively refined the analyzer based on proposal and presentation feedback, especially in the areas of abstraction precision and future extensibility.

AI Statement

AI tools were used during the project as a supplementary aid, primarily for conceptual clarification, debugging discussion, and writing support. In particular, I used GPT 5.3 to:

- discuss abstract interpretation concepts and related work
- reason through Dafny proof failures and proof strategies
- improve the wording of slides, report sections, and explanations
- brainstorm possible pathways for extending the functionality of the the language and increasing the precision of the analyzer

AI was helpful for explanation and brainstorming, especially when I needed alternative proof intuitions or clearer ways to communicate technical ideas. However, the final implementation, proof structure, and submitted code were written, checked, and understood by me. AI suggestions were not accepted blindly and usually required substantial adaptation.

6 Conclusions

This project demonstrated that a small but nontrivial static analyzer can be implemented and formally verified in Dafny. The analyzer successfully detects division-by-zero and uninitialized-variable errors and comes with a machine-checked soundness guarantee.

One of the most interesting aspects of the project was the relationship between abstract interpretation and formal proof. The analyzer itself is conceptually simple: it propagates abstract information through expressions and statements. However, proving that this approximation is sound with respect to concrete execution required much more work than implementing the analyzer itself.

The most difficult part of the project was not writing the analysis rules (which was arguably the simplest part), but structuring the proof so that Dafny could verify them. This required explicit lemmas connecting abstract values to concrete integers and additional proof infrastructure for conditionals and state joining.

If I were to continue the project, the most natural next steps would be:

- extending the language with more features such as while loops, boolean expressions, variable declaration.
- experimenting with different abstract domains such as taint domains, or working with Even/Odd abstract values (relevant for a potential `Pow(base, exponent)` function)
- studying how the project compares more directly to real verified static analyzers such as Verasco [4] and CompCert [6].

Overall, I enjoyed the project and found it to be one of the most intellectually rewarding parts of the course. It provided me with hands-on experience with both static analysis and formal verification, and it also made clear how much proof effort is required to obtain even relatively small correctness guarantees. If I did the project again, I would likely plan the proof structure earlier and isolate helper lemmas sooner, since much of the late-stage effort involved reorganizing proof obligations rather than changing the core analyzer design or expanding language functionality.

References

[1] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM Symposium on Principles of Programming Languages (POPL)*, pages 238–252, 1977.

[2] K. Rustan M. Leino. Dafny: An automatic program verifier for functional correctness. In *Proceedings of the 16th International Conference on Logic for Programming, Artificial Intelligence, and Reasoning (LPAR-16)*, pages 348–370, 2010.

[3] Flemming Nielson, Hanne Riis Nielson, and Chris Hankin. *Principles of Program Analysis*. Springer, 1999.

[4] Jacques-Henri Jourdan, Vincent Laporte, Sandrine Blazy, Xavier Leroy, and David Pichardie. A formally verified C static analyzer. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 247–259, 2015.

[5] Bruno Blanchet, Patrick Cousot, Radhia Cousot, Jérôme Feret, Laurent Mauborgne, Antoine Miné, David Monniaux, and Xavier Rival. A static analyzer for large safety-critical software. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 196–207, 2003.

[6] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009. doi:10.1145/1538788.1538814.